



TheDBAdmin.com

PostgreSQL HA • Training & Practitioner Resources

Patroni Cluster

50 Interview Questions & Answers

A Field Guide for Experienced PostgreSQL DBAs

Author: Versha • Founder, TheDBAdmin.com

Edition 1.0 • May 2026

© 2026 TheDBAdmin.com — All rights reserved.

Index / Subject List

A complete navigation map for this practitioner guide

About this Guide	3
How to Use this Book	3
Visual Conventions	3
Visual Reference (Diagrams at a Glance)	4
Figure 1 — Patroni HA Cluster Architecture	4
Figure 2 — Automatic Failover Workflow	4
Figure 3 — Split-Brain Prevention via Leader Lock	5
Figure 4 — Synchronous vs Asynchronous Replication	5
Part I — Patroni Fundamentals (Q1 – Q10)	7
Part II — Cluster Operations & patronictl (Q11 – Q19)	10
Part III — Configuration & Timing Parameters (Q20 – Q25)	13
Part IV — Failure Scenarios & Recovery (Q26 – Q32)	15
Part V — Replication Modes & WAL Management (Q33 – Q43)	17
Part VI — Troubleshooting Patroni (Q44 – Q49)	20
Part VII — Production Best Practices (Q50)	23
Part VIII — Real-World Scenarios	24
Scenario 1 Primary Down	24
Scenario 2 etcd Down	24
Scenario 3 High Replica Lag	24
Scenario 4 WAL Disk Full	24
Scenario 5 Planned OS Patching	25
Appendix A — Commands Quick Reference	26
Appendix B — References (APA Style)	27

About this Guide

This guide consolidates fifty production-grade interview questions on Patroni-managed PostgreSQL high-availability clusters into a single, classroom-ready reference. It is written for experienced database administrators, site-reliability engineers, and DBRE candidates who need to defend their architectural choices in technical interviews and reason about cluster behaviour under real outages.

Every answer here was validated against the official Patroni documentation maintained by Zalando SE on Read the Docs, against the PostgreSQL 16 documentation, and against the production handbooks of well-known PostgreSQL providers. Where a configurable default is mentioned (for example `ttl = 30 s`, `retry_timeout = 10 s`) the value is drawn directly from the Patroni Settings reference; where a SQL view is shown, the column list matches the corresponding PostgreSQL system catalog page.

How to Use this Book

- Read Parts I to III in order if you are new to Patroni — they build the conceptual scaffolding before any failure scenario is introduced.
- Jump directly to Part VI (Troubleshooting) or Part VIII (Real-World Scenarios) if you are preparing for a live incident drill or an interview that emphasises operational maturity.
- Use Appendix A as a printable cheat sheet for on-call situations — every command listed is safe to copy verbatim into a production terminal.
- Treat each Part as a single training module. The Q&A density (six to eleven questions per part) maps to a 45-minute classroom block at TheDBAdmin.com cadence.

Visual Conventions

Element	Meaning
Q1. What is Patroni?	Numbered interview question. The Q-number is amber; the question is navy.
Answer	The expert-level answer that follows. Written in justified prose for readability.
Code box (monospace, grey)	Validated commands or YAML — safe to run / paste as-is.
Reference table (navy header, alternating rows)	Comparative or lookup content. Always read top-to-bottom.
Figure 1 — caption	Architecture or workflow diagram. Numbered and listed in the Visual Reference page.

Visual Reference

Four diagrams underpin every concept in this guide. They are reproduced here together so the reader has a single visual map to return to while working through the questions. Each diagram is referenced from the relevant answers later in the book.

Figure 1 — Patroni HA Cluster Architecture

A canonical three-node Patroni deployment with a three-node DCS quorum, a routing tier (HAProxy or PgBouncer) and the streaming-replication path between the primary and its two replicas.

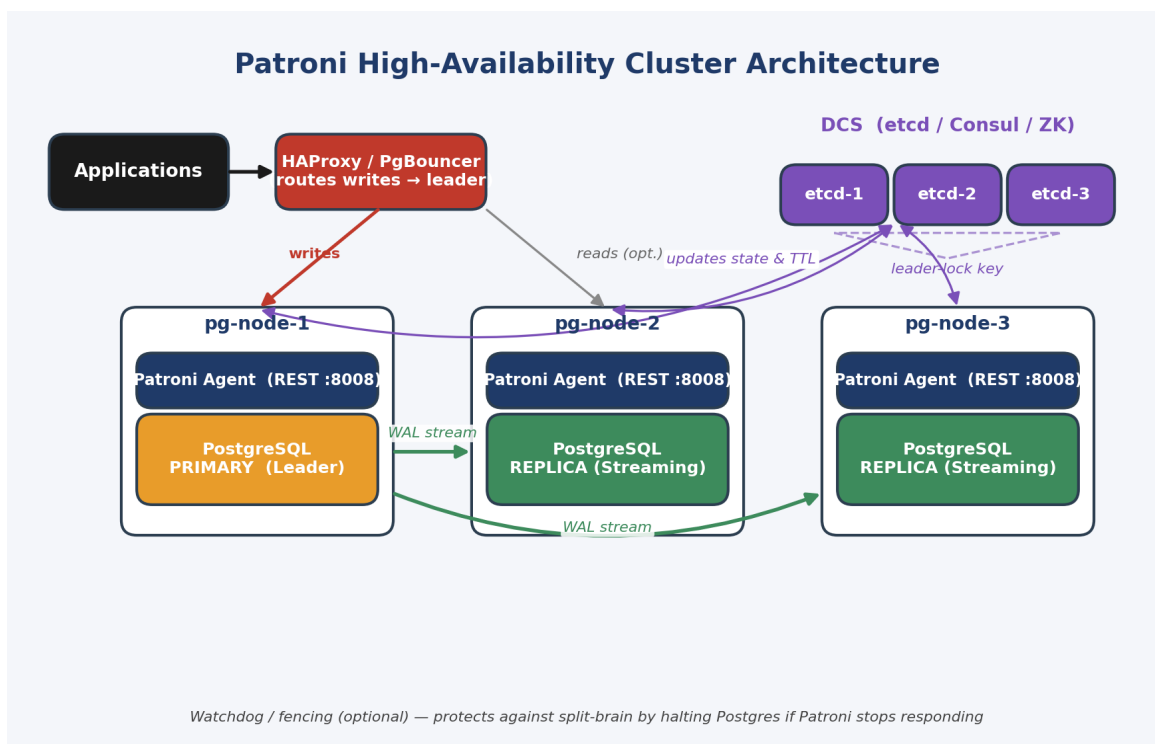


Figure 1 — Patroni HA Cluster Architecture

Figure 2 — Automatic Failover Workflow

The six discrete steps that occur between primary failure and a new primary accepting writes. Read together with the timing rule $\text{loop_wait} + 2 \times \text{retry_timeout} \leq \text{ttl}$.

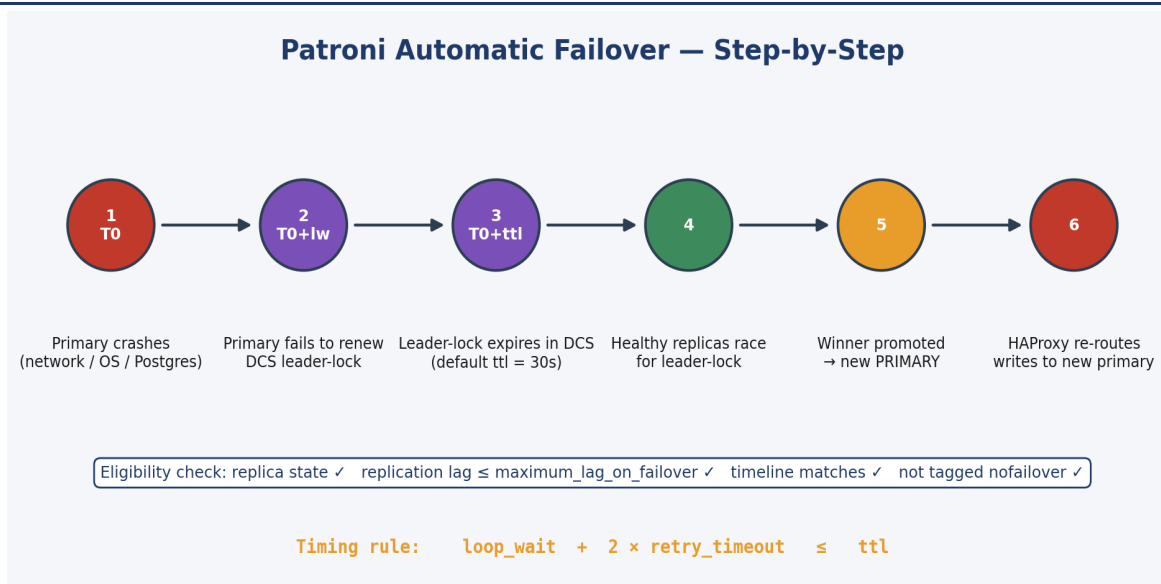


Figure 2 — Automatic Failover Workflow

Figure 3 — Split-Brain Prevention via Leader Lock

A single distributed lock in the DCS is the only authority that determines which node is primary. Without that lock, no PostgreSQL instance accepts writes — which is how Patroni keeps two primaries from ever existing at the same time.

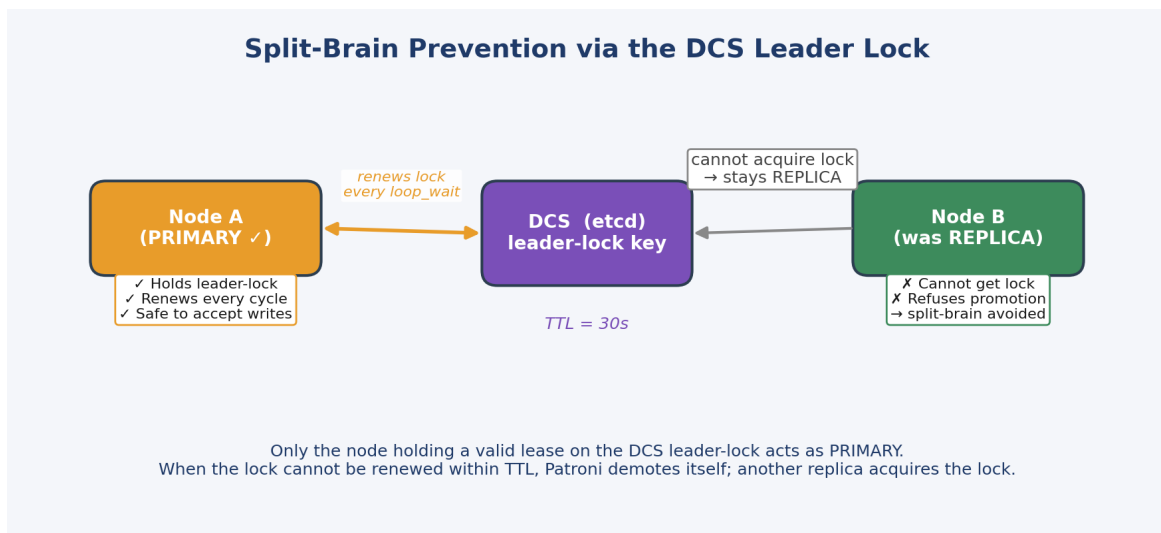


Figure 3 — Split-Brain Prevention via Leader Lock

Figure 4 — Synchronous vs Asynchronous Replication

Side-by-side commit paths. Use this figure to anchor every discussion about RPO (data-loss budget) and commit latency.

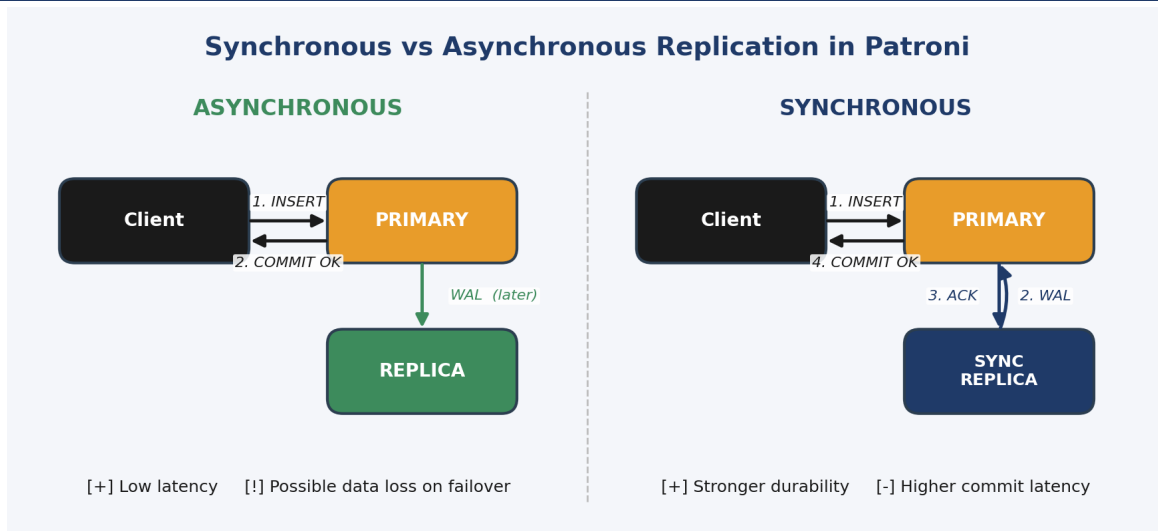


Figure 4 — Synchronous vs Asynchronous Replication in Patroni

Part I — Patroni Fundamentals

These ten foundational questions establish what Patroni actually does, why streaming replication alone is not enough, and which moving parts (PostgreSQL, the Patroni agent, the DCS, and the routing tier) must be present for a real high-availability cluster.

Q1. What is Patroni?

Answer

Patroni is a high-availability framework for PostgreSQL. It runs as an agent next to every PostgreSQL instance in the cluster and is responsible for managing primary and replica nodes, performing automatic failover, supporting planned switchover, and persisting cluster state in a Distributed Configuration Store (DCS) such as etcd, Consul, ZooKeeper or the Kubernetes API. The agent owns the PostgreSQL process: it starts and stops it, edits postgresql.conf, runs pg_rewind, and promotes a standby when DCS confirms it is safe to do so.

Q2. Why do we use Patroni with PostgreSQL?

Answer

PostgreSQL ships with native streaming replication, but the engine itself does not perform automatic failover, leader election or split-brain protection. Patroni layers exactly those missing pieces on top of streaming replication: it elects a leader through a DCS-backed lock, promotes the healthiest eligible replica when the primary dies, manages cluster-wide configuration via a single dynamic-configuration object, and exposes a REST API that load-balancers and monitoring agents can poll to know which node currently accepts writes.

Q3. What are the main components of a Patroni cluster?

Answer

Component	Purpose
PostgreSQL	The database engine itself — primary and one or more streaming replicas.
Patroni agent	Runs on every database node; owns the PostgreSQL process and talks to the DCS.
DCS (etcd / Consul / ZooKeeper / Kubernetes API)	Stores the leader lock, cluster state, member list and dynamic configuration.
Load-balancer (HAProxy / PgBouncer / Pgpool)	Routes application traffic to the current primary (and optionally replicas).
Watchdog (optional)	Fencing safety net — kills PostgreSQL if Patroni stops responding, preventing split-brain.

See Figure 1 (Visual Reference) for how these components fit together in a typical three-node deployment.

Q4. What is DCS in Patroni?**Answer**

DCS stands for Distributed Configuration Store. Patroni uses the DCS to persist cluster state — the leader lock, member entries, last-known timeline, and the dynamic configuration that every node must agree on. Without a quorum-based DCS, Patroni has no safe way to elect a single leader; the leader-lock primitive is what makes split-brain mathematically avoidable.

Q5. Which DCS tools are commonly used with Patroni?**Answer**

Patroni currently supports etcd (v2 and v3), Consul, ZooKeeper, and the Kubernetes API. In production, etcd v3 is the most common choice because it is lightweight, has strong consistency through Raft, and is already operated by most platform-engineering teams (it backs Kubernetes itself). Consul is the second most common choice when an organisation has already standardised on HashiCorp tooling.

Q6. What is the role of etcd in Patroni?**Answer**

etcd stores the Patroni cluster metadata: the leader-lock key, member registration keys, the cluster history of past leaders, and the dynamic configuration. Whichever Patroni agent currently holds the leader-lock key is treated as the primary. If that key cannot be refreshed before its TTL expires, etcd deletes it and the remaining agents race to claim it — that race is what we call an automatic failover.

Q7. What is the leader lock in Patroni?**Answer**

The leader lock is a single, TTL-bound key written into the DCS. By design, only one node in the cluster may hold a valid lock at any moment. The node that holds the lock is the primary; every other node sees the lock owned by someone else and configures itself as a standby. Because the lock has a TTL, a partitioned primary that cannot talk to the DCS will lose ownership automatically once the TTL elapses — preventing two primaries from existing simultaneously.

Q8. What is split-brain?**Answer**

Split-brain is the condition in which two PostgreSQL nodes both believe they are the primary and both accept writes. It is one of the most dangerous failure modes in any HA cluster because each primary diverges with its own committed transactions; when network connectivity is restored, the two timelines are irreconcilable without dropping data. Architecting against split-brain is therefore the first design goal of any HA tool, including Patroni.

Q9. How does Patroni prevent split-brain?**Answer**

Patroni delegates the split-brain question to the DCS by way of a single TTL-bound leader-lock key. Only the node that currently owns that key may run PostgreSQL as primary; any other node must run as a read-only replica. If the current primary loses network access to the DCS, it cannot refresh the lock, so the lock expires and Patroni demotes that PostgreSQL instance (or, with a watchdog, fences the whole machine). At the same time, no replica can be promoted until it successfully acquires the lock. The result is mutual exclusion guaranteed by the DCS quorum, not by any individual node. See Figure 3 in the Visual Reference for an illustration.

Q10. What is the difference between PostgreSQL streaming replication and Patroni?

Answer

PostgreSQL Streaming Replication	Patroni
Replicates WAL from primary to one or more standbys.	Manages the entire HA lifecycle around streaming replication.
No built-in failover — a human must promote a standby manually.	Performs automatic failover when the primary fails.
No leader election; no concept of cluster state.	Holds cluster state and leader lock in a DCS quorum.
No application-facing health endpoint.	Exposes a REST API on port 8008 for load-balancers and monitors.
Each node's postgresql.conf is edited locally.	Manages a single dynamic configuration applied across all members.

Part II — Cluster Operations & patronictl

Patroni exposes a command-line tool, `patronictl`, that is the day-one interface for every DBA running a Patroni cluster. Part II covers the commands you will use during routine operations and during planned maintenance windows — and the conceptual distinctions (leader vs primary, replica vs standby, failover vs switchover) that determine which command is the right one to reach for.

Q11. What is `patronictl`?

Answer

`patronictl` is the command-line client for managing a Patroni cluster. With it you can list the current cluster topology, trigger a planned switchover, force a failover, reinitialise a broken replica, edit the dynamic configuration, view the failover history, and pause or resume automatic HA actions. Every command takes the same `-c` flag pointing at the Patroni YAML configuration of the node you are operating from.

```
# List cluster members and their current roles
patronictl -c /etc/patroni/patroni.yml list
```

Q12. How do you check Patroni cluster status?

Answer

Use `patronictl list`. It queries the DCS for the leader-lock owner, the member states and the replication lag of each standby. A healthy three-node cluster looks like this:

```
patronictl -c /etc/patroni/patroni.yml list

+ Cluster: pg-cluster -----+-----+-----+
| Member | Host           | Role   | State   |
+-----+-----+-----+-----+
| pg1    | 192.168.1.101 | Leader | running |
| pg2    | 192.168.1.102 | Replica | streaming |
| pg3    | 192.168.1.103 | Replica | streaming |
+-----+-----+-----+-----+
```

Q13. What is the difference between leader and primary?

Answer

Inside the Patroni model, leader is the node that currently owns the DCS leader-lock — the cluster-wide source of truth. Primary is a PostgreSQL term: it means the database instance that is out of recovery and accepting writes. Under normal operation these are the same node. They can diverge transiently during failover (the old primary may still be alive on the network but has already lost the lock, so it is no longer the leader).

Q14. What is the difference between replica and standby?

Answer

Both terms describe a PostgreSQL instance in recovery mode that is replaying WAL from the primary. Patroni and patronictl prefer the word replica; the PostgreSQL documentation prefers the word standby. They are interchangeable, and the role reported by patronictl list is Replica.

Q15. What is failover in Patroni?

Answer

Failover is the unplanned promotion of a replica to primary, triggered when the current primary stops being healthy. Patroni performs failover automatically (after the leader-lock expires) but you can also force one from the command line — for example, when a primary is hung in a way the agent has not yet detected.

```
patronictl -c /etc/patroni/patroni.yml failover
```

Q16. What is switchover in Patroni?

Answer

Switchover is a planned, graceful role change: the current primary is cleanly shut down (or demoted) and a specific replica is promoted. It is the right command for routine maintenance — OS patching, kernel upgrades, hardware replacement — because it produces zero data loss when followed correctly.

```
patronictl -c /etc/patroni/patroni.yml switchover
```

Q17. What is the difference between failover and switchover?

Answer

Failover (unplanned)	Switchover (planned)
Triggered by primary failure or by force.	Triggered by an operator during maintenance.
May complete with the primary unreachable.	Old primary is healthy and is gracefully demoted first.
Risk of data loss in asynchronous mode.	Zero data loss when executed during quiet traffic.
Used during outages.	Used during planned change windows.

Q18. What happens during automatic failover?

Answer

When the primary stops responding, the Patroni agent on that node also stops renewing the DCS leader-lock. The lock expires after the configured TTL. The remaining Patroni agents discover the missing lock during their next HA loop, check that they themselves are eligible (in-state, lag within `maximum_lag_on_failover`, timeline matches, not tagged `nofailover`) and the winner of the race writes itself into DCS as the new leader and promotes its local PostgreSQL. The whole sequence is shown step-by-step in Figure 2.

Q19. How does Patroni select a replica for promotion?**Answer**

Patroni evaluates each replica against a set of eligibility checks: the PostgreSQL state must be running, the replica must be streaming, the replication lag in bytes must be at or below `maximum_lag_on_failover`, the timeline must match the primary, and the member must not carry the `nofailover` tag. Among the eligible candidates, the one with the most advanced WAL position (lowest lag) wins. If no replica passes the checks, Patroni refuses to promote — preserving correctness over availability.

Part III — Configuration & Timing Parameters

The behaviour of a Patroni cluster under failure is dictated almost entirely by four numbers: `ttl`, `loop_wait`, `retry_timeout`, and `maximum_lag_on_failover`. Together they decide how fast a failover happens, how aggressive the lock-refresh cycle is, and how much replication lag the cluster will tolerate before promoting a stale replica. This Part walks each one and the timing rule that ties them together.

Q20. What is `maximum_lag_on_failover`?

Answer

`maximum_lag_on_failover` is the upper bound, in WAL bytes, on how far a replica can be behind the last known leader position and still be considered eligible for promotion during an automatic failover. The default is 1 MB (1,048,576 bytes). Setting it too high accepts data loss; setting it too low can leave the cluster with no eligible promotion candidate.

```
# In the Patroni dynamic configuration
maximum_lag_on_failover: 1048576 # 1 MB – replicas lagging more than 1 MB are not
promoted
```

Q21. What is `ttl` in Patroni?

Answer

`ttl` is the time-to-live (in seconds) of the leader-lock key in the DCS. The current primary must renew the key before this interval elapses. If renewal fails — because the primary crashed, lost its network or stopped its Patroni agent — the key disappears from the DCS and the failover process begins. Patroni's default is 30 seconds; the minimum allowed is 20.

```
ttl: 30
```

Q22. What is `loop_wait`?

Answer

`loop_wait` is the number of seconds Patroni sleeps between successive iterations of its HA loop. On every iteration the agent re-reads cluster state from the DCS, checks PostgreSQL health, refreshes its own member key, and acts on any pending action. Default is 10 seconds. A shorter `loop_wait` means faster reaction at the cost of more DCS traffic.

```
loop_wait: 10
```

Q23. What is `retry_timeout`?

Answer

`retry_timeout` is the maximum time (seconds) Patroni will spend retrying transient DCS or PostgreSQL operations before treating them as failures. A short network blip below this value is tolerated and does not cause the leader to demote itself. Default is 10 seconds; the minimum is 3.

```
retry_timeout: 10
```

Q24. What is the relationship between `ttl`, `loop_wait`, and `retry_timeout`?

Answer

Patroni enforces a safety rule whenever you edit these three values together:

```
loop_wait + 2 * retry_timeout <= ttl  
  
Defaults: 10 + 2 * 10 = 30,    ttl = 30    -> valid
```

The rule guarantees that, within a single TTL window, the agent has time to complete at least one HA loop and one full set of retries before being declared dead. Violating the rule produces a configuration error; tightening the numbers gives you a faster failover but a less forgiving cluster.

Q25. What happens if DCS goes down?

Answer

When the DCS quorum is lost, no Patroni agent can safely change the cluster topology. The current primary continues serving traffic for as long as it can confirm it is the legitimate leader; depending on the `dcf_failsafe_mode` setting and the watchdog, it may also self-demote to protect against split-brain. No failover, switchover, or member registration can occur until DCS is healthy again — which is why a three-node DCS quorum is non-negotiable in production.

Part IV — Failure Scenarios & Recovery

Part IV walks through the failures that actually happen in production: a primary that crashes, a replica that diverges, a node that needs to be removed cleanly, and the SQL views that tell you what state the cluster is in right now. Every command and view in this Part is validated against the PostgreSQL 16 documentation.

Q26. What happens if the primary crashes?

Answer

Patroni stops being able to refresh the leader-lock on behalf of the dead primary. The lock expires in DCS after the configured TTL. The other Patroni agents promote the healthiest eligible replica and write themselves into DCS as the new leader. The routing layer (HAProxy, PgBouncer, Pgpool) detects the new primary via the REST API health check and starts forwarding writes to the new node. Application reconnects are handled by the connection pooler or the application driver.

Q27. What happens if a replica crashes?

Answer

The primary continues to accept writes. Patroni marks the failed replica as stopped or in error in `patronictl` list and stops streaming WAL to it. When the node is brought back, Patroni will attempt to rejoin it — using `pg_rewind` if possible, or a fresh base backup if the timelines have diverged. Useful first-line checks:

```
systemctl status patroni
journalctl -u patroni -f
```

Q28. How do you reinitialise a failed replica?

Answer

The `reinit` subcommand rebuilds a replica from scratch using the configured `create_replica_method` (typically `basebackup`, `pgBackRest` or `WAL-G`). This is the right tool when `pg_rewind` has failed or the data directory is corrupt.

```
patronictl -c /etc/patroni/patroni.yml reinit <cluster_name> <node_name>
```

Q29. How do you remove a dead node from Patroni?

Answer

First confirm the node is gone — stop the Patroni service, decommission the host, and verify it no longer appears in any monitoring. Then remove its DCS keys with `patronictl remove`:

```
patronictl -c /etc/patroni/patroni.yml remove <cluster_name>
```

The command prompts for a confirmation token. For partial cleanup of stale member keys you can also use dcs-specific tools (etcdctl del, consul kv delete) — but only after verifying that the node is no longer running, otherwise it will simply re-register itself.

Q30. How do you check PostgreSQL replication status?

Answer

On the primary, query `pg_stat_replication`, which reports one row per connected standby including the WAL positions it has sent, written, flushed and replayed:

```
SELECT client_addr,
       state,
       sync_state,
       sent_lsn,
       write_lsn,
       flush_lsn,
       replay_lsn
FROM   pg_stat_replication;
```

On the replica, query `pg_stat_wal_receiver`, which reports the upstream connection state and the WAL position the replica has received from the primary:

```
SELECT * FROM pg_stat_wal_receiver;
```

Q31. How do you check whether a node is primary or replica?

Answer

The simplest, version-independent way is to call `pg_is_in_recovery()`. It returns true on a replica (still in recovery mode) and false on a primary.

```
SELECT pg_is_in_recovery();
```

Output	Meaning
false	Primary — accepting writes.
true	Replica — in recovery, read-only.

Q32. How do you check replication lag?

Answer

On the replica, time-based lag is the difference between `now()` and the timestamp of the last replayed transaction:

```
SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;
```

On the primary, byte-based lag against each connected standby is computed with `pg_wal_lsn_diff()` against `pg_current_wal_lsn()`:

```
SELECT client_addr,  
       pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes  
FROM   pg_stat_replication;
```

Part V — Replication Modes & WAL Management

Choosing between synchronous and asynchronous replication is the single most consequential RPO decision a DBA makes. Part V covers both modes, the PostgreSQL parameters that make replication possible at all, and the WAL-management primitives (replication slots, `archive_command`, `wal_keep_size`) that decide whether a lagging replica can catch up or has to be rebuilt from scratch.

Q33. What is synchronous replication in Patroni?

Answer

Under synchronous replication, a transaction COMMIT on the primary does not return success to the client until at least one designated synchronous standby has confirmed that the WAL has been received (or written, or flushed, depending on `synchronous_commit`). Patroni manages the `synchronous_standby_names` list automatically so that the elected synchronous replica is always tracked in the DCS. This mode trades commit latency for a much smaller data-loss budget — typically zero RPO in the steady state.

Q34. How do you enable synchronous mode in Patroni?

Answer

Synchronous mode is enabled in the dynamic configuration, not in `postgresql.conf`. The three keys that matter are `synchronous_mode` (turn the feature on), `synchronous_mode_strict` (refuse to commit if no sync standby is available) and `synchronous_node_count` (how many standbys must acknowledge).

```
# Dynamic configuration (edit via patronictl edit-config)
synchronous_mode: true
synchronous_mode_strict: false
synchronous_node_count: 1
```

```
patronictl -c /etc/patroni/patroni.yml edit-config
```

Q35. What is the difference between synchronous and asynchronous replication?

Answer

Synchronous	Asynchronous
Commit waits for the sync standby ACK.	Commit returns as soon as WAL is local.
Lower RPO — typically zero in steady state.	Possible data loss on failover.
Higher and more variable commit latency.	Lower, more predictable commit latency.
Sensitive to network and standby health.	Tolerant of standby outages — they just lag.

See Figure 4 for the side-by-side commit-path comparison.

Q36. Can Patroni failover in asynchronous mode?

Answer

Yes. Patroni will promote an asynchronous replica as long as its lag is within `maximum_lag_on_failover` and the eligibility checks pass. The implicit cost is that any transactions committed on the primary but not yet shipped to the chosen replica are lost. This is the correct trade-off for many workloads (analytics, content sites, queue back-ends) but it must be a conscious choice — never the default.

Q37. How can you reduce data loss during failover?**Answer**

- Enable synchronous replication for the workloads where RPO must be zero.
- Tune `maximum_lag_on_failover` down so a stale replica is never silently promoted.
- Use physical replication slots so the primary never recycles WAL a standby still needs.
- Configure continuous WAL archiving (`archive_command`, `pgBackRest`, WAL-G) so missing WAL can be replayed during recovery and PITR is possible.
- Monitor replication lag, `archive_command` success and disk headroom, and alert on each.
- Right-size the network and storage between primary and standby — most lag is not a Postgres problem but an I/O or NIC problem.

Q38. What is a replication slot?**Answer**

A replication slot is a persistent registration on the primary that records the minimum WAL position still required by a downstream consumer. The primary will refuse to recycle WAL segments newer than that position, so a temporarily disconnected replica can always catch up — at the cost of WAL retention on the primary.

Q39. What is the risk of inactive replication slots?**Answer**

If a slot stays inactive — typically because the consumer has been removed or is broken — the primary keeps every WAL segment newer than the slot's `restart_lsn`. WAL accumulates indefinitely, fills `pg_wal`, and eventually crashes the primary with a disk-full error. Always monitor `pg_replication_slots` and remove slots whose owner is gone.

```
SELECT slot_name, active, restart_lsn
FROM   pg_replication_slots;
```

Q40. Why is `wal_level = replica` required?**Answer**

`wal_level` controls how much information PostgreSQL writes into the WAL. At `wal_level = minimal` only crash recovery is supported. At `replica`, the WAL includes the data needed for physical replication, hot standbys, point-in-time recovery and archiving. Patroni requires at least `replica` because streaming

replication is its substrate. (logical adds row-level changesets for logical decoding but is not required for Patroni HA.)

```
postgresql:
  parameters:
    wal_level: replica
```

Q41. Which PostgreSQL parameters matter most for Patroni replication?

Answer

```
wal_level:          replica
max_wal_senders:    10          # one per standby + headroom for backups
max_replication_slots: 10
hot_standby:        on          # allow read queries on standbys
wal_keep_size:      1024MB      # safety net even with replication slots
archive_mode:       on
archive_command:    '...'       # pgBackRest / WAL-G / cp to NFS
```

Q42. What is max_wal_senders?

Answer

max_wal_senders caps the number of concurrent WAL-sender processes the primary will spawn. Each connected streaming standby uses one; each base backup taken via pg_basebackup uses one; logical replication consumers use one each. A common starting value is 10 — enough for three replicas, two backup jobs, and headroom for ad-hoc operations.

Q43. What is max_replication_slots?

Answer

max_replication_slots caps the total number of replication slots (physical plus logical) the primary will allow. Like max_wal_senders, it must be at least the number of consumers you intend to support; setting it equal to max_wal_senders is a sensible default.

Part VI — Troubleshooting Patroni

Patroni rarely fails on its own — what usually fails is the substrate underneath it (PostgreSQL, the DCS, the disk, the network). This Part is a structured triage guide: where to look first, which logs to read, and which symptoms tell you which subsystem is at fault.

Q44. Which logs do you check during a Patroni issue?

Answer

Always read three layers of logs together — Patroni, PostgreSQL, and the DCS — because a single failure often surfaces in all three:

```
# Patroni
journalctl -u patroni -f
tail -f /var/log/patroni/patroni.log

# PostgreSQL
journalctl -u postgresql -f
tail -f /var/lib/pgsql/16/data/log/postgresql*.log

# DCS (etcd shown; substitute consul/zookeeper as needed)
journalctl -u etcd -f
```

Q45. A replica is not joining the cluster. How will you troubleshoot?

Answer

- Verify the Patroni service is running on the replica node (`systemctl status patroni`).
- Check ownership and mode of the PostgreSQL data directory (must be `postgres:postgres` and `0700`).
- Confirm the replication user password matches what is in `pgpass` / the Patroni YAML.
- Verify a replication entry exists in `pg_hba.conf` for the replica's IP.
- Test TCP reachability from replica to primary on 5432 (`pg_isready -h <primary_ip>`).
- Confirm the replica can talk to the DCS (`curl etcd / consul / zk`).
- Check whether the primary has recycled the WAL the replica needs — look for replication-slot misses or `wal_keep_size = 0`.
- Look at the base-backup logs (basebackup or `pg_basebackup` output) for I/O or auth failures.
- If timelines have diverged, run `patronictl reinit` to rebuild from a fresh basebackup.
- Verify disk headroom on the replica before basebackup starts.

```
systemctl status patroni
journalctl -u patroni -f
pg_isready -h <primary_ip> -p 5432
```

Q46. A Patroni node is showing stopped. What will you check?

Answer

stopped in patronictl list means Patroni cannot keep PostgreSQL running. Triage in this order:

- Service status and last 100 journal lines.
- PostgreSQL data-directory existence, ownership and mode.
- postgresql.conf parse errors (postgres --check-config).
- Port conflicts on 5432 / 8008.
- Permission issues on /var/lib/pgsql or the WAL directory.
- DCS reachability.
- Disk-full on the data, WAL or log volume.
- A failed recovery from the last crash (look for FATAL in postgresql.log).
- Misconfigured Patroni YAML — for example, a non-existent create_replica_method.

```
systemctl status patroni
journalctl -u patroni -n 100
```

Q47. PostgreSQL is running but Patroni says the node is unhealthy. Why?**Answer**

Patroni assesses health both through libpq and through its own REST API. A divergence between PostgreSQL state and Patroni state is almost always one of these:

- The REST endpoint on :8008 is not reachable from inside the host (firewall, bind address).
- PostgreSQL is responding slowly or has a broken authentication (Patroni cannot log in as the superuser).
- Replication lag has exceeded maximum_lag_on_failover, so Patroni marks the replica as not eligible.
- The timeline does not match the leader's — the replica has diverged.
- DCS communication is broken, so Patroni cannot read the cluster state to compare.
- The PostgreSQL process is alive but Patroni did not start it (e.g., started manually outside Patroni).

```
curl http://localhost:8008/health
curl http://localhost:8008/patroni
```

Q48. The primary crashed but failover did not happen. What will you check?**Answer**

- Is the DCS quorum healthy? Failover cannot proceed if etcd / Consul lost quorum.
- Are any replicas eligible? If every replica exceeds maximum_lag_on_failover, no promotion happens.
- Is the cluster in pause mode? Pause disables all automatic HA actions.

- Are the replicas tagged nofailover? They are explicitly excluded from promotion.
- Did the watchdog or fencing layer fire and reboot a node mid-promotion?
- Was there a network partition that left the old primary isolated but technically alive (TTL not yet expired)?

```
patronictl -c /etc/patroni/patroni.yml list
patronictl -c /etc/patroni/patroni.yml show-config
```

Q49. What is Patroni pause mode?

Answer

Pause mode tells Patroni to stop driving the cluster — no failover, no switchover, no automatic reinit. PostgreSQL keeps running, but Patroni becomes a passive observer. This is the right setting for high-risk maintenance such as kernel upgrades, storage migration, or a hands-on debug session where you do not want the agent reacting to your manual changes.

```
# Enter pause mode
patronictl -c /etc/patroni/patroni.yml pause

# Resume normal HA actions
patronictl -c /etc/patroni/patroni.yml resume
```

Part VII — Production Best Practices

Q50. What is the production checklist for a Patroni cluster?

Answer

A production-ready Patroni deployment is the result of disciplined choices in seven areas. Treat the table below as a go-live gate: every row must be marked done before the cluster carries customer traffic.

Area	Checklist item
DCS	Three-node etcd or Consul cluster on separate failure domains. Quorum loss = no failover.
PostgreSQL	Streaming replication confirmed; wal_level = replica; replication slots in use.
Patroni	YAML reviewed; REST API reachable from the load-balancer and from monitoring.
Load balancer	HAProxy / PgBouncer / Pgpool configured to read the Patroni health endpoint.
Failover	Automatic failover tested end-to-end at least once before go-live.
Switchover	Planned-maintenance switchover rehearsed; runbook printed and signed off.
Backup	pgBackRest or WAL-G configured; backups restored at least once into a sandbox.
Monitoring	Prometheus + Grafana dashboards in place; alerts on lag, lock loss, archive failure, disk.
Security	TLS between Patroni and DCS; firewall rules; strong, rotated replication user password.
Disaster recovery	PITR procedure tested; RTO / RPO documented; off-site backup verified.

Part VIII — Real-World Scenarios

Each scenario below is a five-minute interview prompt. The answers are written the way an experienced DBA would speak in a war-room — short, ordered, and grounded in the SQL and shell commands that actually verify what is happening.

Scenario 1 — Primary down

Question: The primary node is down. What happens?

Answer

Patroni detects that the primary is no longer renewing the DCS leader-lock. After the TTL elapses, the lock disappears and the remaining agents race for it. The replica with the lowest lag (and not tagged `nofailover`) wins, is promoted to primary, and registers itself as the new leader. The load-balancer reads the updated REST health endpoint and routes writes to the new primary. The old primary, when it returns, is rejoined as a replica via `pg_rewind` (or rebuilt via `basebackup` if `rewind` is not possible).

Scenario 2 — etcd down

Question: `etcd` is down. What is the impact?

Answer

Without a healthy DCS quorum, Patroni cannot safely read or update the leader-lock. The current primary continues to serve traffic for a short window, but no failover, no switchover, and no member registration can occur until DCS is restored. If `dcf_failsafe_mode` is enabled, the primary may self-demote after the TTL to avoid an undetected split-brain. The immediate operational priority is to restore `etcd` quorum — do not attempt to reconfigure Patroni while the DCS is down.

Scenario 3 — Replica lag is high

Question: A replica is lagging. What will you check?

Answer

- Network latency and packet loss between primary and replica.
- Disk I/O saturation on the replica (`iostat`, `pg_stat_io`).
- CPU saturation on the replica (single WAL-replay process is single-threaded).
- WAL generation rate on the primary (a write storm can outpace any replica).
- Long-running transactions on the replica that block recovery.
- Replication-slot status — is the slot active?
- Archive-restore delay if the replica fell back to `restore_command`.
- Vacuum and checkpoint pressure on the replica.

```
SELECT * FROM pg_stat_replication;  
SELECT * FROM pg_replication_slots;
```

Scenario 4 — WAL disk full

Question: The WAL directory is filling up. What can the reason be?

Answer

- Inactive replication slot retaining WAL forever.
- A replica down for an extended period and the wal_keep_size still being honoured.
- archive_command failing silently — pg_wal cannot be recycled until the segment is archived.
- Unusually high WAL generation from a bulk load or VACUUM FULL.
- A long-running transaction holding back vacuum and forcing WAL retention.
- A stuck backup process keeping a slot open.

```
SELECT slot_name, active, restart_lsn FROM pg_replication_slots;  
SELECT * FROM pg_stat_archiver;
```

Scenario 5 — Planned OS patching

Question: How do you patch the current primary server?

Answer

Use a switchover, never a failover, for planned work. The whole procedure is:

```
# 1. Confirm cluster state before any change  
patronictl -c /etc/patroni/patroni.yml list  
  
# 2. Hand the lease to a chosen replica  
patronictl -c /etc/patroni/patroni.yml switchover  
  
# 3. Once another node is primary, patch the old primary  
systemctl stop patroni  
# ...apply OS updates...  
reboot  
systemctl start patroni  
  
# 4. Verify everything came back as Replica streaming  
patronictl -c /etc/patroni/patroni.yml list
```

Appendix A — Commands Quick Reference

Every command in this appendix has been validated against the official Patroni and PostgreSQL documentation. They are safe to copy directly into a production terminal.

Cluster operations

```
patronictl -c /etc/patroni/patroni.yml list
patronictl -c /etc/patroni/patroni.yml switchover
patronictl -c /etc/patroni/patroni.yml failover
patronictl -c /etc/patroni/patroni.yml reinit <cluster_name> <node_name>
patronictl -c /etc/patroni/patroni.yml show-config
patronictl -c /etc/patroni/patroni.yml edit-config
patronictl -c /etc/patroni/patroni.yml pause
patronictl -c /etc/patroni/patroni.yml resume
```

Service & log inspection

```
systemctl status patroni
journalctl -u patroni -f
curl http://localhost:8008/patroni
curl http://localhost:8008/health
```

PostgreSQL replication & state

```
SELECT pg_is_in_recovery();

SELECT * FROM pg_stat_replication;

SELECT * FROM pg_stat_wal_receiver;

SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;

SELECT slot_name, active, restart_lsn FROM pg_replication_slots;
```

Appendix B — References

References are presented in American Psychological Association (APA) 7th edition style. All URLs were verified at the time of writing.

Zalando SE. (n.d.). Patroni — A template for PostgreSQL HA with ZooKeeper, etcd, or Consul. Read the Docs.

<https://patroni.readthedocs.io/>

Zalando SE. (n.d.). Patroni configuration settings. Read the Docs.

https://patroni.readthedocs.io/en/latest/yaml_configuration.html

Zalando SE. (n.d.). patronictl command-line reference. Read the Docs.

<https://patroni.readthedocs.io/en/latest/patronictl.html>

Zalando SE. (n.d.). Patroni REST API reference. Read the Docs.

https://patroni.readthedocs.io/en/latest/rest_api.html

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation — Chapter 27: High availability, load balancing, and replication. <https://www.postgresql.org/docs/16/high-availability.html>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation — pg_stat_replication.

<https://www.postgresql.org/docs/16/monitoring-stats.html#MONITORING-PG-STAT-REPLICATION-VIEW>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation — Replication slots.

<https://www.postgresql.org/docs/16/warm-standby.html#STREAMING-REPLICATION-SLOTS>

etcd Authors. (2024). etcd documentation. <https://etcd.io/docs/>

HashiCorp. (2024). Consul documentation. <https://developer.hashicorp.com/consul/docs>

Crunchy Data. (2024). PostgreSQL high availability with Patroni — best-practice guide.

<https://access.crunchydata.com/documentation/postgres-operator/>

End of Guide

© 2026 TheDBAdmin.com • www.thedbadmin.com